# Student Registry API: Start & Stop Commands

Stage 1 (local development) on Windows | PostgreSQL 18 local, Kafka in Docker, app via Maven

## 1. The key idea: three separate pieces

`mvn spring-boot:run` starts **only your Spring Boot app**. It does **not** start Kafka or PostgreSQL. Each piece is started and stopped on its own, and the app simply connects to the other two when it needs them.

| Piece | Where it runs | Started by | Port | Must be up for |
|---|---|---|---|---|
| PostgreSQL | Windows service (installed) | Windows, automatically at boot | 5432 | Every endpoint |
| Docker Desktop | Windows app | You (Start menu) | - | Kafka (it runs inside Docker) |
| Kafka | Docker container named **kafka** | `docker compose up -d` | 29092 | POST (create student) |
| Spring Boot app | Windows (Java) | `mvn spring-boot:run` | 8080 | - |

**Start order:** PostgreSQL > Docker Desktop > Kafka > App.
**Stop order:** the reverse: App > Kafka > (Docker Desktop) > (PostgreSQL).
Why: the app connects to PostgreSQL and Kafka when it starts, so they should be ready first. Stopping the app first avoids error messages while it shuts down.

## 2. Daily quick reference

Open PowerShell in the project folder. PostgreSQL normally starts by itself with Windows, so most days you only need these steps.

### Start everything

```
# 1. Start Docker Desktop from the Start menu and wait for "Engine running"
cd "C:\Users\Pranav Joshi\OneDrive\Documents\Determination\student-registry"
docker compose up -d          # 2. start Kafka
docker compose ps             #    wait until STATUS says (healthy), about 20 seconds
mvn spring-boot:run           # 3. start the app; wait for "Started StudentRegistryApplication"
```

### Stop everything

```
Ctrl + C                      # 1. in the terminal running the app
docker compose stop           # 2. stop Kafka (keeps its messages)
# 3. optional: quit Docker Desktop from the system tray to free memory
```

## 3. PostgreSQL (local Windows service)

Windows service name: `postgresql-x64-18`. Its start type is **Automatic**, so it is already running whenever your PC is on. You rarely need to touch it.

> **Start, stop and restart need an Administrator PowerShell.** Right-click PowerShell > Run as administrator. Checking the status works in a normal window.

| Action | Command (PowerShell) |
|---|---|
| Check status | `Get-Service postgresql-x64-18` |
| Start (Admin) | `Start-Service postgresql-x64-18` |
| Stop (Admin) | `Stop-Service postgresql-x64-18` |
| Restart (Admin) | `Restart-Service postgresql-x64-18` |
| Using the GUI | Press Win + R, type `services.msc`, find **postgresql-x64-18**, right-click Start / Stop / Restart |

### Verify you can connect (and see the data)

```
$psql = "C:\Program Files\PostgreSQL\18\bin\psql.exe"
$env:PGPASSWORD = "student123"
& $psql -U student -h localhost -d studentdb -c "SELECT * FROM students;"
```

If the app runs while PostgreSQL is stopped, every endpoint fails with an error (the app cannot open a database connection).

## 4. Docker Desktop

Kafka lives inside Docker, so Docker Desktop must be running before any `docker` command works.

| Action | How |
|---|---|
| Start | Start menu > **Docker Desktop**. Wait until the bottom-left shows **Engine running**. |
| Check it is ready | `docker info` (prints details; an error means it is not running yet) |
| Stop | System tray (near the clock) > right-click the whale icon > **Quit Docker Desktop**. This also stops the Kafka container. |

> Error **failed to connect to the docker API ... dockerDesktopLinuxEngine** means Docker Desktop is not started. Start it and wait about a minute.

## 5. Kafka (Docker container)

Run these from the **student-registry** folder, because `docker compose` reads `docker-compose.yml` from the current folder.

| Action | Command | What it does |
|---|---|---|
| Start (first time or after down) | `docker compose up -d` | Downloads the image if needed, creates and starts the container. `-d` = run in background. |
| Check status | `docker compose ps` | Shows Up / (healthy). Wait for (healthy) before starting the app. |
| Stop (keep messages) | `docker compose stop` | Pauses the container. Messages are kept. |
| Start again after stop | `docker compose start` | Resumes the same container with its messages. |
| Restart | `docker compose restart` | Stop + start in one step. |
| Remove (delete messages) | `docker compose down` | Deletes the container and all Kafka messages. Next `up -d` gives a fresh, empty Kafka. |
| Watch Kafka's log | `docker compose logs -f kafka` | Live log. Ctrl + C stops watching (Kafka keeps running). |

### stop vs down

| | docker compose stop | docker compose down |
|---|---|---|
| Container | Kept (stopped) | Deleted |
| Kafka messages in student-events | Kept | Lost |
| Bring it back with | `docker compose start` or `up -d` | `docker compose up -d` |
| Use when | Ending your day | You want a clean slate |

### Look inside Kafka

```
# List topics
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list

# Read every message in student-events (Ctrl + C to exit)
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh `
  --bootstrap-server localhost:9092 --topic student-events --from-beginning

# How far the app's consumer has read (LAG 0 = fully caught up)
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh `
  --bootstrap-server localhost:9092 --describe --group student-registry-group
```

The backtick ` at the end of a line continues the command on the next line in PowerShell.

## 6. Spring Boot app

| Action | Command | Notes |
|---|---|---|
| Start (from source) | `mvn spring-boot:run` | Compiles and runs. Ready when the log shows **Started StudentRegistryApplication**. |
| Stop | Ctrl + C | In the same terminal. Press Y if asked to terminate the batch job. |
| Build a JAR | `mvn package -DskipTests` | Creates target\student-registry-0.0.1-SNAPSHOT.jar |
| Start from the JAR | `java -jar target\student-registry-0.0.1-SNAPSHOT.jar` | Faster start; no compiling. |
| Run from VS Code | Open StudentRegistryApplication.java > **Run** above main() | Stop with the red square in the debug toolbar. |

### Check it is running

```
Invoke-RestMethod http://localhost:8080/api/students
```

### Error: "Port 8080 was already in use"

Another copy of the app is still running (for example one started from VS Code). Stop that one, or find and stop whatever is using port 8080:

```
Get-NetTCPConnection -LocalPort 8080 -State Listen      # shows OwningProcess (the PID)
Stop-Process -Id (Get-NetTCPConnection -LocalPort 8080 -State Listen).OwningProcess -Force
```

## 7. Check the status of everything at once

```
Get-Service postgresql-x64-18 | Select-Object Name, Status       # PostgreSQL: Running?
docker compose ps                                               # Kafka: (healthy)?
Get-NetTCPConnection -LocalPort 8080 -State Listen -EA SilentlyContinue   # App: anything listed?
```

## 8. What happens if one piece is down while the app runs

The two Kafka rows were tested on this project. The 2-minute limit is Kafka's default retry setting.

| Situation | GET / PUT / DELETE | POST (create student) |
|---|---|---|
| Kafka stops while the app is running | Work normally | 201 returned at once. The event waits in the app's memory and is sent when Kafka returns (within about 2 minutes, otherwise it is dropped and an error is logged). |
| Kafka already down when the app starts | Work normally | App still starts. POST waits 60 seconds, then 500 "Send failed". The insert is rolled back, so no student is saved. |
| PostgreSQL down | Fail (500) | Fail (500) |
| Docker Desktop quit | Work normally | Same as the Kafka rows above (Kafka runs inside Docker) |

While Kafka is down the app log repeats **Connection to node ... could not be established**. This is the app retrying, and it stops once Kafka is back.